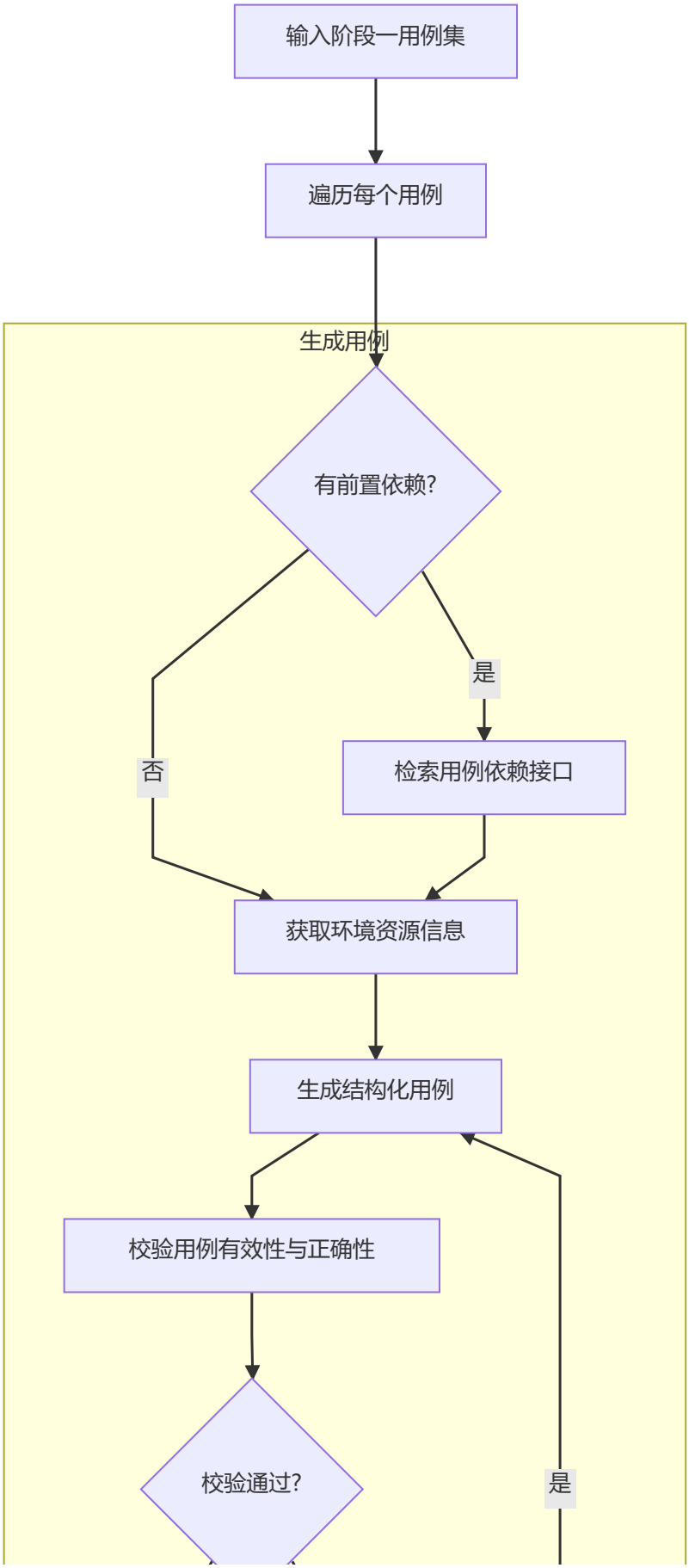
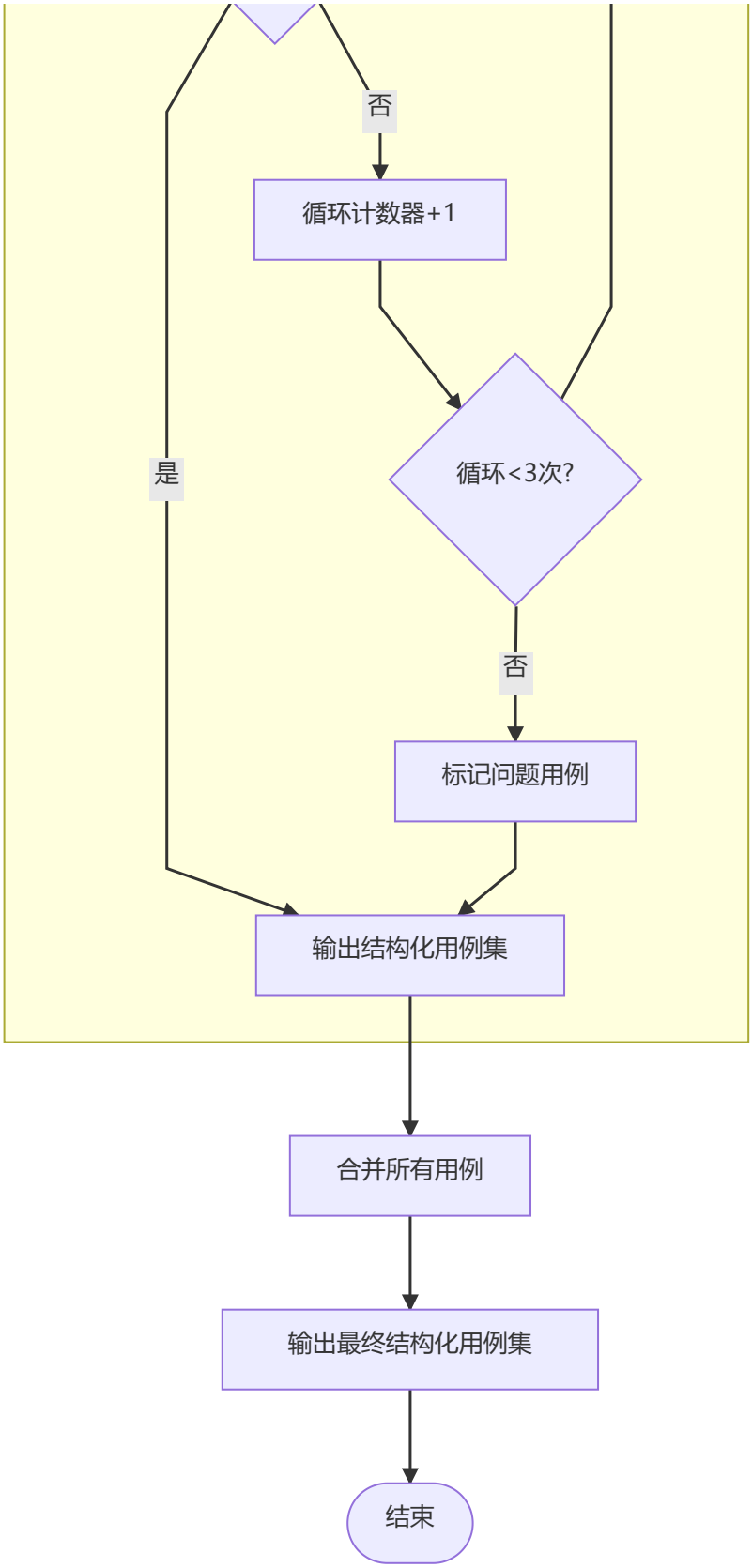
02-09、接口自动化Agent实战之结构化用例生成流程图

1、流程图设计





2、结构化用例生成提示词

```
# @Author  : 木森
# @weixin: python771
# 结构化用例生成提示词
from langchain.prompts import PromptTemplate
```

```
structured_case_prompt = PromptTemplate(
    input_variables=['output_format',
                     'case_info',
                     'case_api',
                     'other_api',
                     'test_data',
                     'files_list',
                     'function_list',
                     'additional_info'
                     ],
    template=r"""
你是一位资深的接口测试专家，精通 HTTP 协议、RESTful API 设计、JSON 数据结构和测试用例编写规范。你同时具备将复杂测试需求结构化表达的能力，能够高效生成标准化、高质量的自动化测试用例。

任务目标：根据用户提供的测试用例信息和接口文档，生成符合指定结构的标准化接口测试用例，输出内容应完全符合下方结构规范。


## 一、生成测试用例时的详细规则说明：

1. 参数来源分析原则
请严格分析主接口中每个参数的来源：
- **分析前置依赖返回的字段说明和当前用例主请求参数字段说明是否符合**
- **严格要分析字段数据是否引用前置前置依赖接口返回资源的id，避免遗漏**
- **有些接口参数定义不规范项目，引用某个资源的id,会省略id值，比如引用project_id,在参数定义的时候为project,引用user_id，参数定义为user**
- **需要前置接口获取** → 在 `preconditions` 中定义提取规则，并使用变量引用的语法**
- **分析接口请求的鉴权类型和鉴权信息，鉴权令牌的提取和引用**
- **分析接口请求的鉴权类型和鉴权信息，鉴权字段的提取和使用**


2. 依赖参数识别方法
特别注意以下通常来自前置接口的参数类型：
- **认证令牌**：比如:token,  access_key, authorization
- **关联数据的id**：user_id, order_id, project_id, file_id
- **状态/步骤值**：status, step, phase, progress
- **关联对象**：reference_id, parent_id, related_id


3. **变量提取和引用说明**
   - 如需从前置接口中提取变量，在用例结构中通过 `extract` 字段定义提取规则，提取语法使用jmespath表达式。例如：
     ```json
     "extract": [
         ["字段名称", "jmespath表达式"]
     ]
     ```
   - 被提取变量引用时请使用 `{{{{变量名}}}}` 格式，例如：
     ```json
     "headers": {{ "Authorization": "Bearer {{{{token}}}}" }}
     ```


4. **测试数据的变量化引用规范**
   - 所有测试数据中提供的字段，如 `username` 和 `password`，请统一使用双大括号格式引用，例如：
     ```json
     "body": {{
         "user": "{{{{username}}}}",
```
```

```
        "password": "{{{{password}}}}"
    }}
    ```
  - 测试执行准备的测试数据如下：
      {test_data}

5. **multipart/form-data 类型接口的处理方式**
   - 若请求类型为 `multipart/form-data`，请将正文数据放入 `files` 字段，`body` 字段设置为空对象 `{{}}`，结构如下：
     ```json
     "body": {{}},
     "files": {{
         "pic": ["文件名", "文件路径", "文件类型"],
         "name": "张三",
         "age": 18
     }}
     ```
   - 所有文件字段的值必须从以下文件列表中选择：
     {files_list}

6. **动态数据与加密处理字段的函数引用规范**
   - 对于不可重用字段（如手机号、邮箱等）或需动态生成的字段，请调用用户提供的函数，使用 `$函数名(参数=值)` 语法
   - 如果函数的参数值，是某个引用的变量则使用变量的引用方式表示 `$函数名(参数={{{{变量}}}})`

     例如：
     ```json
     "body": {{
         "username": "$generate_username(prefix=t)",
         "email": "$generate_email(prefix={{{{my_email}}}})",
         "mobile": "$generate_mobile()"
     }}
     ```
   - 可用的函数列表如下：
     {function_list}

7. **数据库结构说明**
   - 如用户未提供数据库表结构，请将用例结构中的 `database` 字段设置为空列表：`[]`

8. **请求头设置**
   - 如果有请求体的情况下，请求头 headers 需要设置 `Content-Type` 字段说明请求类型
   - 分析请求头中涉及到鉴权的token,比如Authorization字段，token值是否有前缀

9. **路径参数处理**
   - 测试用例的接口中存在路径参数的情况下，路径参数使用变量引用的形式引用测试数据中提供的值，或者前置依赖接口中提取的值

10. **base_url 设置**
   - 对于测试用例的 base_url，如果测试环境中没有提供相关的值，则使用变量引用的语法表示

二、用户提供的输入信息：

1. **测试用例基础信息**
{case_info}

2. **当前测试用例对应接口文档**
```
```

```
{case_api}

3. **该用例依赖的前置接口文档**
{other_api}

---

## 三、补充说明：

    {additional_info}

## 四、输出要求：
请将结果以标准JSON格式输出，所有字段名和字符串都必须用双引号。输入结构如下
{output_format}

- 输出格式为 JSON（非代码块，不带 markdown 标记）
- 所有字段值、变量引用、函数调用必须符合上方规则
- 不要遗漏任何结构字段，即使为空也要补全
- 不进行说明或解释，仅输出测试用例结构化 JSON 内容
- 输出的测试用例结构规范（必须遵循）：

"""
)

# 输出数据的格式要求
format_output = {
    "name": "用例名称",
    "description": "用例描述",
    "project": "所属项目",
    "module": "所属模块",
    "interface": "接口名称",
    "preconditions": [
        {
            "name": "前置步骤名称",
            "request": {
                "interface_id": "接口ID",
                "method": "HTTP方法",
                "url": "接口路径",
                "base_url": "基础URL",
                "headers": {"头信息"},
                "params": {"查询参数"},
                "body": {"请求体为表单/json类型时的请求体参数"},
                "files": {"请求体类型为multipart/form-data时的参数"}
            },
            # 数据提取
            "extract": [
                ["字段名称", "接口返回的响应字段路径(jmespath提取表达式)", ]
            ]
        }
    ],
    "request": {
        "interface_id": "主接口ID",
        "name": "主请求名称",
        "method": "HTTP方法",
        "url": "接口路径",
        "base_url": "基础URL",
        "headers": {"请求头信息"},
        "params": {"查询参数"},
```

```
        "body": {"请求体"}
    },

    "assertions": {
        "response": [
            {
                "type": "断言类型(equal/not_null/http_code),http_code为响应状态码",
                "field": "响应字段路径(jmespath提取表达式)，当类型为http_code时，该字段为,HTTP状态码",
                "expected": "预期值"
            }
        ],
        "database": [
            {
                "query": "SQL查询语句",
                "expected": {"字段": "预期值"}
            }
        ]
    }
}
```

3、输出用例数据结构

```
# 输出数据的格式要求
format_input = {
    "name": "用例名称",
    "description": "用例描述",
    "project": "所属项目",
    "module": "所属模块",
    "interface": "接口名称",
    "preconditions": [
        {
            "name": "前置步骤名称",
            "request": {
                "interface_id": "接口ID",
                "method": "HTTP方法",
                "url": "接口路径",
                "base_url": "基础URL",
                "headers": {"头信息"},
                "params": {"查询参数"},
                "body": {"请求体为表单/json类型时的请求体参数"},
                "files": {"请求体类型为multipart/form-data时的参数"}
            },
            # 数据提取
            "extract": [
                ["字段名称", "接口返回的响应字段路径(jmespath提取表达式)", ]
            ]
        }
    ],
    "request": {
        "interface_id": "主接口ID",
        "name": "主请求名称",
        "method": "HTTP方法",
        "url": "接口路径",
        "base_url": "基础URL",
        "headers": {"请求头信息"},
```

```
        "params": {"查询参数"},
        "body": {"请求体"}
    },

    "assertions": {
        "response": [
            {
                "type": "断言类型(equal/not_null/http_code),http_code为响应状态码",
                "field": "响应字段路径(jmespath提取表达式)，当类型为http_code时，该字段为,HTTP状态码",
                "expected": "预期值"
            }
        ],
        "database": [
            {
                "query": "SQL查询语句",
                "expected": {"字段": "预期值"}
            }
        ]
    }
}
```

4、资源检测工具

1、随机数据生成和数据加密工具获取

```
import inspect
import mimetypes
import os
from typing import List, Dict


def get_class_all_functions(cls) -> List[Dict]:
    """获取测试数据生成和数据加密所有函数的信息"""
    functions = []
    # 获取类中所有方法
    for name, method in inspect.getmembers(cls, predicate=inspect.isfunction):
        # 获取方法的签名
        sig = inspect.signature(method)
        # 获取参数信息，排除self参数
        params = {}
        for param_name, param in sig.parameters.items():
            if param_name != 'self':
                params[param_name] = param.default if param.default is not param.empty else None
        # 获取方法文档
        desc = inspect.getdoc(method) or ""
        # 添加方法信息
        functions.append({
            "method_name": name,
            "desc": desc,
            "params": params
        })
    return functions
```

```python
if __name__ == '__main__':
    print(get_class_all_functions(get_class_all_functions))
```

2、上传文件接口可用测试文件获取工具

```python
def get_file_type(file_path: str) -> str:
    """
    获取文件类型
    Args:
        file_path: 文件路径
    Returns:
        str: 文件类型描述
    """
    # 初始化mimetypes
    mimetypes.init()
    # 获取MIME类型
    mime_type, _ = mimetypes.guess_type(file_path)
    if mime_type is None:
        # 如果无法检测到MIME类型，则返回文件扩展名
        ext = os.path.splitext(file_path)[1][1:].lower()
        return ext if ext else "unknown"
    return mime_type


def inspect_env_data() -> List[Dict]:
    """
    获取data/files目录下的文件信息
    Returns:
        List[Dict]: 包含文件名、文件路径和文件类型的列表
    """
    current_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    file_info_list = []
    file_dir = os.path.join(current_dir, 'data', 'files')

    # 确保目录存在
    if not os.path.exists(file_dir):
        os.makedirs(file_dir, exist_ok=True)
        return file_info_list

    # 遍历目录下的所有文件
    for file_name in os.listdir(file_dir):
        file_path = os.path.join(file_dir, file_name)
        if os.path.isfile(file_path):  # 只处理文件，不处理子目录
            file_info = {
                "file_name": file_name,
                "file_path": file_path,
                "file_type": get_file_type(file_path)
            }
            file_info_list.append(file_info)

    return file_info_list
```

5、数据解析类的定义

```python
from langchain_core.output_parsers import JsonOutputParser

class CaseModel(BaseModel):
    name: str = Field(..., description="用例名称", max_length=100)
    description: Optional[str] = Field(None, description="用例描述")
    project: str = Field(..., description="所属项目")
    module: str = Field(..., description="所属模块")
    interface: str = Field(..., description="接口名称")
    preconditions: List = Field(default_factory=list, description="前置条件列表")
    request: Dict = Field(..., description="主请求参数")
    assertions: Dict = Field(..., description="断言配置")

parser = JsonOutputParser(pydantic_schema=CaseModel)
```

6、结构化用例生成流程

```python
# @Author  : 木森
# @weixin: python771
"""
单条用例生成的流程实现
"""
from langgraph.graph import StateGraph
from langgraph.constants import START, END
from typing import TypedDict, List

from pydantic import BaseModel, Field
from langchain_core.output_parsers import JsonOutputParser
from agents.models import deepseek_v3_llm
from api_auto_test.common.inspect_env import inspect_class_functions,
inspect_file_data
from api_auto_test.main import run_test
from prompts.case_generator.structured_case_generator import output_format,
structured_case_prompt
from rag.data_storage.sql_storage import SQLDB


class StateNode(TypedDict):
    """定义节点的状态数据"""
    project: str  # 项目名称
    case_api: dict  # 用例接口
    case_info: dict  # 基础用例信息
    additional_info: str  # 额外的说明信息
    other_api: list  # 前置条件依赖的接口详情
    test_data: str  # 测试数据(放在测试环境里面)
    files_list: list  # 文件上传的用例可用的文件列表
    function_list: list  # 生成测试数据和加密处理的可用的函数列表
    output_case: dict  # 最终输出的用例
    test_report: dict  # 验证用例可用性得到的测试报告
    is_available: bool  # 用例是否可用
    generator_count: int  # 用例生成的次数


class CaseParser(BaseModel):
    """结构化用例数据提取类"""
    name: str = Field(..., description='用例名称')
    description: str = Field('无', description='用例描述')
```

```
    project: str = Field('', description='项目名称')
    module: str = Field('', description='所属模块')
    interface: str = Field('', description='接口名称')
    preconditions: List = Field(default_factory=list, description='前置条件')
    request: dict = Field(default_factory=dict, description='请求体')
    assertions: dict = Field(default_factory=dict, description='断言信息')


class ApiCaseStructuredGeneratorFlow:
    """接口用例结构化生成的工作流"""

    @staticmethod
    def check_dependency(state: StateNode):
        """检测用例前置依赖接口"""
        case_info = state.get("case_info")
        dependencies = case_info.get('dependencies', [])
        if dependencies:
            # 有前置条件和依赖的接口(检索出依赖接口详细信息)
            return '获取依赖接口'
        else:
            # 无前置条件,获取用例生成的可用环境数据
            return '获取测试环境数据'

    @staticmethod
    def get_dependency_apis(state: StateNode):
        """获取依赖接口"""
        sql = 'SELECT * from api_interface WHERE project=%s AND summary  IN %s'
        case_info = state.get("case_info")
        dependencies = case_info.get('dependencies', [])
        with SQLDB() as db:
            db.cursor.execute(sql, (state.get('project'), dependencies))
            other_api = db.cursor.fetchall()
            print("前置依赖接口数据如下: ", other_api)
            return {'other_api': other_api}

    @staticmethod
    def get_test_env_data(state: StateNode):
        """获取测试环境数据"""
        # 获取当前的测试环境中提供的测试数据
        test_data = state['test_data']
        # 获取测试数据随机生成的可用函数
        functions = inspect_class_functions()
        # 获取文件上传参数的可用文件列表
        files = inspect_file_data()
        return {'test_data': test_data, 'function_list': functions,
'files_list': files}

    @staticmethod
    def structure_case_generation(state: StateNode):
        """结构化用例生成"""
        # 准备提示词
        # output_format
        # 准备数据解析器
        parser = JsonOutputParser(pydantic_schema=CaseParser)
        # 构建调用链
        chain = structured_case_prompt | deepseek_v3_llm | parser
        # 生成用例
        response = chain.invoke({
```

```
            "output_format": output_format,
            "case_info": state.get("case_info", ''),
            "case_api": state.get("case_api", ''),
            "other_api": state.get("other_api", ''),
            "test_data": state.get("test_data", ''),
            "function_list": state.get("function_list", ''),
            "files_list": state.get("files_list", ''),
            "additional_info": state.get("additional_info", '')

        })
        print("生成的测试用例如下：", response)
        return {'output_case': response}

    @staticmethod
    def check_case(state: StateNode):
        """
        验证用例的可用性和正确性
        可用性：最直接的方式：运行该用例看运行过程是否出现错误
        :param state:
        :return:
        """
        # 获取生成的用例
        case = state.get("output_case")
        # 执行生成用例
        report = run_test([case])
        print("测试用例执行结果如下:", report)
        if report['summary'].get('passed') | report['summary'].get('failed'):
            return {'is_available': True}
        else:
            # 获取用例生成的次数
            generator_count = state.get('generator_count', 0)
            return {"test_report": report, 'is_available': False,
'generator_count': generator_count + 1}

    # 判断用例生成的次数
    @staticmethod
    def check_generator_count(state: StateNode):
        """判断用例生成的次数"""
        generator_count = state.get('generator_count', 0)
        if generator_count > 3:
            return '标记为问题用例'
        else:
            return '重新生成用例'

    def create_workflow(self):
        """创建工作流graph"""
        graph = StateGraph(StateNode)
        graph.add_node('检测用例前置依赖接口', self.check_dependency)
        graph.add_node('获取依赖接口', self.get_dependency_apis)
        graph.add_node('获取测试环境数据', self.get_test_env_data)
        graph.add_node('结构化用例生成', self.structure_case_generation)
        graph.add_node('验证用例的可用性', self.check_case)

        # 流程编排
        graph.add_conditional_edges(START, self.check_dependency, ['获取依赖接口',
'获取测试环境数据'])
        graph.add_edge('获取依赖接口', '获取测试环境数据')
        graph.add_edge('获取测试环境数据', '结构化用例生成')
```

```
        graph.add_edge('结构化用例生成', '验证用例的可用性')
        graph.add_edge('验证用例的可用性', END)

        return graph.compile()


if __name__ == '__main__':
    case_info = {'name': '正常创建项目-参数正确',
                 'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/',
                           '请求体包含正确的name和user参数'],
                 'expected': ['HTTP状态码为201', '响应体包含id、name、user_id和
create_time字段', 'name值与请求参数一致'],
                 'dependencies': ['用户登录']}
    case_api = {'version': None, 'description': '创建项目', 'path':
'/api/pro/projects/', 'tag': None,
                'responses': '{"201": {"id": {"type": "integer", "description":
"项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id":
{"type": "integer", "description": "项目负责人"}, "create_time": {"type":
"string", "description": "创建时间"}}, "422": {"detail": {"type": "array",
"description": ""}}}',
                'summary': '创建项目', 'method': 'POST', 'module': None,
                'requestBody': '{"name": {"type": "string", "description": "项目
名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
                'project': 'web测试平台', 'parameters': '[]', 'distance':
-3.5762786865234375e-07}
    project = 'web测试平台'

    app = ApiCaseStructuredGeneratorFlow().create_workflow()
    response = app.invoke({"case_info": case_info,
                           "case_api": case_api,
                           "project": project,
                           "test_data": {
                               "username": "python771",
                               "password": "musen12399",
                               "base_url": "http://124.222.144.182:8888"
                           }}, )
```